Définition de types et Pointeurs

1 Exercices – *no more global variables!*

1. Lors du TP précédent, vous avez codé la version itérative de l'algorithme d'Euclide étendu, en utilisant de variables globales pour contourner le problèmes des fonctions qui ne retournent qu'une seule valeur. Cependant, même si utiles, l'utilisation des variables globales doit se limiter au stricte minimum !
    a. Vous coderez donc une 2ème version de votre fonction « eea() », appelée « eea2() » qui n'utilisera plus des variables globales. Vous modifierez au minimum possible le code original pour faire que votre fonction soit testée avec le code « EEAres res = eea2(254, 44); printf("u:%d,v:%d,gcd:%d\n",res.u,res.v,res.gcd); ». EEAres est le nom défini pour une structure composé de 3 éléments entiers : u, v et gcd. EEAres sera défini globalement dans le programme, mais aucune variable globale de ce type ne doit exister.
2. Créer une structure pour retourner plusieurs valeurs depuis une fonction est une solution pratique. Cependant, ceci implique parfois dupliquer pendant quelques instants l'utilisation de la mémoire (e.g. on a eu 2 variables EEAres dans notre code, l'un dans la fonction eea2() et l'autre dans la fonction main()). S'il s'agissait des données dans l'ordre de grandeur des gigaoctest, sa duplication aurait un cout prohibitif ! Dans ce cas-là, au lieu de dupliquer l'information, on préfère passer l'adresse mémoire où se trouve nos données et que la modification de ces dernières soient faites « sur place ». Vous devez prouver donc que vous êtes capable de manipuler directement la mémoire de l'ordinateur pour traiter vos données.
    a. Créez une fonction « eea3() » dont le prototype est le suivant « int eea3(int a, int b, int *u, int *v); ». Votre fonction retourne le pgcd(a,b), mais les coefficients sont écrits dans les adresses pointées par « u » et « v ». Aucune variable globale ne doit exister dans votre programme (gcd, u et v seront déclarées dans la fonction main()).
3. C est un langage assez laxiste. Par exemple, comme nous l'avons vu en cours, le prototype de la fonction main() peut être écrit de plusieurs manières. Dites si « int main(int argc, char argv[][]) » est un prototype valable pour main() et pourquoi. N'hésitez pas à écrire un court programme en C pour compiler et tester ce prototype.
4. Écrire un programme C qui reçoit plusieurs arguments par la ligne de commande. Votre programme doit afficher sur écran chaque argument reçu dans une nouvelle ligne. Cependant, si votre programme reçoit un argument (ou paramètre) « -r », les arguments suivants sont montrés dans l'ordre inverse, jusqu'à l'apparition d'un nouveau « -r ». Pour comparer 2 chaines de caractères vous pouvez utiliser la fonction « strcmp() ». Exemples de sortie de votre programme,
    a. "$ ./showargs first" donne la sortie :
    2 arguments where provided
    Arg[0]: ./showargs
    Arg[1]: first
    b. "$ ./ showargs first -r test arg1" donne la sortie :
    5 arguments where provided
    Arg[0]: ./showargs

Arg[1]: first
==> Reversing display
Arg[3]: tset
Arg[4]: 1gra

c. "./showargs first -r test arg1 -r test" donne la sortie :
7 arguments where provided
Arg[0]: ./showargs
Arg[1]: first
==> Reversing display
Arg[3]: tset
Arg[4]: 1gra
==> Reversing display
Arg[6]: test

5. Utilisez votre programme de l'exercice précédent pour expliquer
   a. quelle différence existe entre un programme exécuté sous la forme « `$ ./prog arg1 arg2` » et un programme exécuté de la façon suivante : « `$ ./prog "arg1 arg2"` »
   b. Que se passe-t-il donc si le nom du fichier exécutable est renommé ? est-ce que l'argument 0 (zéro) reflète le nouveau nom du fichier exécutable ?

2 Exercices - presque un Triangle de Pascal

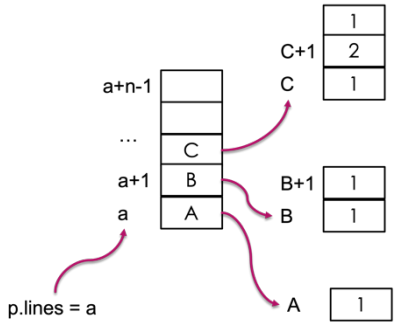
6. Le triangle de Pascal est une tableau triangulaire des coefficients binomiaux. La partie la plus haute du triangle correspond à la valeur $\binom{0}{0} = 1$, et les prochaines lignes n et colonnes k peuvent être calculés avec la formule $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$, pour tout $n > 0$ et $0 \leq k \leq n$. Sans utiliser donc la notation factoriel $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ (et donc uniquement avec des sommes), écrire une fonction C appelée createPascalTriangle() qui créera un triangle de Pascal à n lignes. Voici quelques précisions/étapes pour réussir votre programme :
   a. Votre fonction createPascalTriangle() renvoie zéro si tout se passe bien et que le triangle est créé. -x en cas d'erreur (la valeur x sera définie plus tard).
   b. Votre fonction recevra en argument un pointeur de type Ptriangle ainsi que un entier n. Ptriangle est un type défini d'une structure composée d'un entier « size » et un pointeur vers des pointeurs de valeurs int appelé « lines » (« int size; int **lines; »).
   c. Codez votre fonction, mais à ce stade, écrivez uniquement le code qui permet de tester la bonne valeur de n, et d'écrire sur « size » de votre pointeur Ptriangle la valeur de n. Si n est plus petit que 1, renvoyez -1 et « size » n'est jamais modifié.
   d. Avant de compiler et tester, écrivez également une fonction dont le prototype est « void showPascalTriangle(Ptriangle p) ; », mais que pour l'instant ne fait qu'afficher la taille du Triangle. Exemple : « Size: 3 ».
   e. Une fois les 2 fonctions écrites avec le minimum de code, testez-les avec le code :

```
Ptriangle p1;
```

```
p1.size = 0; p1.lines = NULL;
int res = createPascalTriangle(&p1, 0); // n = 0
if (res != 0) {
   printf("Triangle not created\n");
   exit(res);
}
showPascalTriangle(p1);
```

ce que devrait afficher « Triangle not created ». Changeant la valeur de n à 1 ou 5, votre programme devrait afficher « Size: 1 » ou « Size: 5 » selon le cas.

f. Si tout se passe bien, ajoutez à « createPascalTriangle() » le code qui permet de réserver, avec malloc(), n pointeurs vers des pointeurs des int et assignez-le à « lines » du pointeur Ptriangle. En effet, votre triangle de Pascal sera composé de n lignes (ou chaque ligne est composée d'un nombre variables des valeurs int). Puis, par une boucle, vous réservez avec malloc() l'espace pour écrire 1 int, si on est dans le première slot des n pointeurs, 2 int's si on est dans le 2ème slot des n pointeurs, et ainsi successivement. Graphiquement vous êtes en train de construire un triangle de la manière suivante :



g. Après avoir compilé votre code et si tout va bien, ajoutez le code nécessaire pour tester si malloc() a réussi à réserver la mémoire. Si malloc() échoue (et retourne la valeur NULL), retournez la valeur -2.

h. Compilez votre programme. Si tout va bien, essayez d'écrire la valeur 1 sur chaque espace de mémoire réservée pour les int.

i. Compilez votre programme et modifiez cette fois-ci la fonction « showPascalTriangle() » pour montrer chaque élément du triangle de Pascal. Commencez par tester votre code avec

n = 1	n = 2	n = 3
Size: 1 1	Size: 2 1 1 1	Size: 3 1 1 1 1 1 1

3 Exercices - debugger un programme C

Très probablement, pour réussir l'exercice précédent, vous avez dû imaginer les zones de mémoire et la manière dont cette dernière était modifiée à fur et à mesure que le code s'exécutait. L'algorithme que vous venez de mettre en place n'est pas compliqué et si quelque chose n'allait pas, le problème venait probablement d'un pointeur ou déréférence mal utilisé.

Savoir inspecter un programme peut s'avérer très utile dans cette situation (et dans plusieurs autres !). Cette tâche-là est effectué à l'aide d'un débuggeur. Nous allons profiter du code précédent pour apprendre à débugger un programme C.

7. Compilez votre programme avec l'option « -g » pour produire un exécutable appelé « tpascald » compatible avec le debuggeur GDB
8. Exécutez votre programme avec GDB : « $ gdb tpascald ». Après cette commande, le débuggeur s'exécute mais pas votre programme « tpascald ».
    a. Si tout se passe bien, on obtient un *prompt* « (gdb) » à partir duquel on peut debugger notre programme.
9. Dites à GDB d'exécuter votre programme : exécutez « run » dans le *prompt* GDB.
10. GDB a exécuté votre programme, du début à la fin sans s'arrêter. Pas très utile tout ça ! Souvent, si quelque chose ne va pas dans notre programme, nous avons en tête un certain morceau de code qui nous pose de doutes. Supposons qu'à partir de la ligne « p1.size = 0; p1.lines = NULL; » nous ne savons pas si notre code est correcte. On souhaite donc arrêter l'exécution de notre programme avant l'exécution de cette ligne et inspecter notre code :
    a. Trouvez le numéro de ligne grâce à votre éditeur de texte.
    b. Dites à gdb d'exécuter votre programme mais de s'arrêter à ce numéro cde ligne (supposons que ça soit la ligne 16) : « (gdb) break 16 »
    c. Exécutez à nouveau votre programme avec « run ».
    d. Si vous voyez « Breakpoint…. », « 16 p1.size = 0; …. », félicitation ! vous êtes sur le point de débugger votre programme correctement !
11. On voudrait voir le contenu de « p1 » avant l'initialisation (ligne 16 de l'exercice 10).
    a. Exécutez « print p1.size » pour voir la valeur de « size » : « (gdb) print p1.size »
    b. Exécutez « print p1.lines » pour voir la valeur de « lines ».
    c. Quels sont les valeurs de « size » et « lines » avant intialisation ?
12. Pour exécuter la ligne 16 et aller à la 17, exécutez « next »
    a. Vérifiez le contenu de « size » et « lines » à nouveau. Quels sont ses valeurs cette fois-ci ?
13. Exécutez « next » à nouveau. Notez que votre debuggeur est passé à la ligne 18 (« if (res != 0) { ») ou lieu de vous montrer l'exécution de la fonction « createPascalTriangle() » !
    a. Faisons en sorte de visiter la fonction « createPascalTriangle() » pour vérifier son fonctionnement.
    b. Exécutez « continue » pour que votre programme finisse son exécution sans s'arrêter.
    c. Effacé le « breakpoint » à la ligne 16 (« delete 1 », 1 c'est le numéro de breakpoint qu'on souhaite effacer)
    d. Créez un breakpoint à la ligne 17 et exécutez votre programme.
    e. Maintenant, au lieu d'exécuter « next », exécutez « step » et vérifiez que vous êtes bien à l'intérieur de la fonction « createPascalTriangle() »
14. Maintenant que vous savez comment arrêter et analyser votre programme, faite le nécessaire pour donner les valeurs de « a », « a+1 », « A », « B » pour un test « n = 2 ». Quelques astuces :
    a. Vous utilisez « info break » pour listes les breakpoints

b. « print lines » donne la valeur de « lines » (donc une adresse). Et « print *lines » donne le contenu de l'adresse gardée dans « lines ».
c. Si « lines » est un tableau, gdb reconnaitra l'arithmétique des pointeurs (ex. « print lines+1 », « print *(lines+1)+1 »).
d. « print lines[1] » != « print lines + 1 », mais « print &lines[1] » == « print lines+1 ». Pourquoi ?

15. Optionnel. Exercice « master des pointeurs ». Donnez le contenu de « B+1 » et le contenu de « A » pour un test « n = 2 ».

4 Exercices - finissons notre Triangle de Pascal

16. Si tout va bien, modifiez votre programme pour mettre en place la formule du triangle de Pascal donnez dans l'exercice 6. Validez votre programme en faisant plusieurs tests.